



Systèmes de recommandation

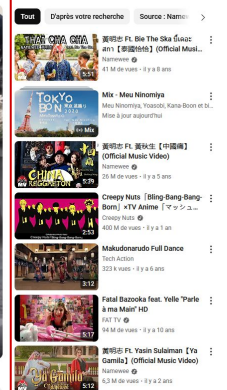
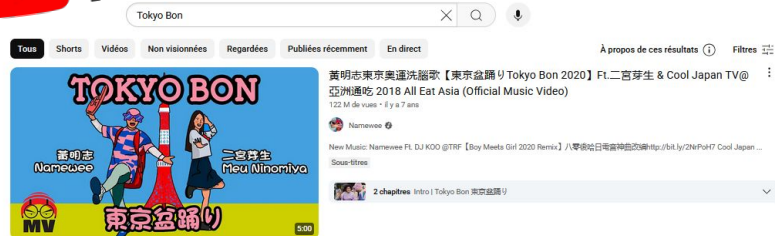
Léonard Benedetti et Guillaume Nollet

De quoi on parle ?

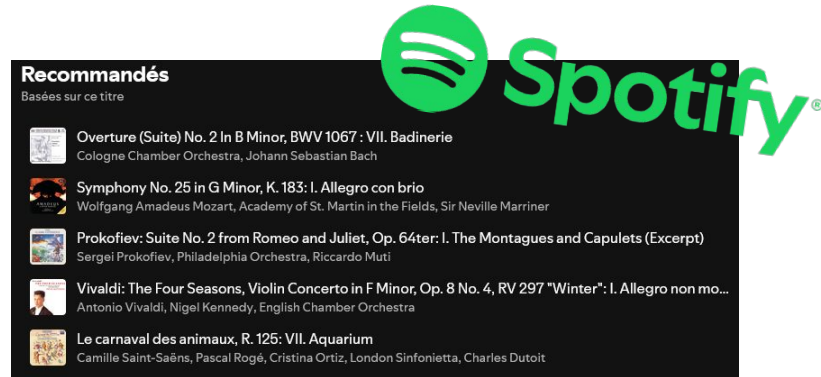
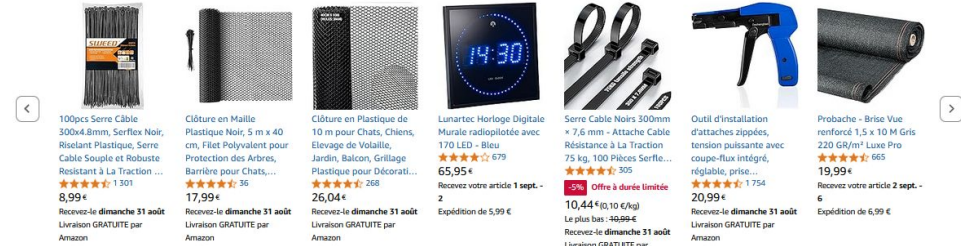


- Un **système de recommandation** est un algorithme qui vise à **sélectionner des éléments** (e.g. livres, films, articles, etc.) qui sont les plus susceptibles **d'intéresser ou d'être pertinents** pour un utilisateur.
- C'est un cas particulier de **filtrage de l'information**.

De quoi on parle ?



Les clients ayant acheté cet article ont également acheté

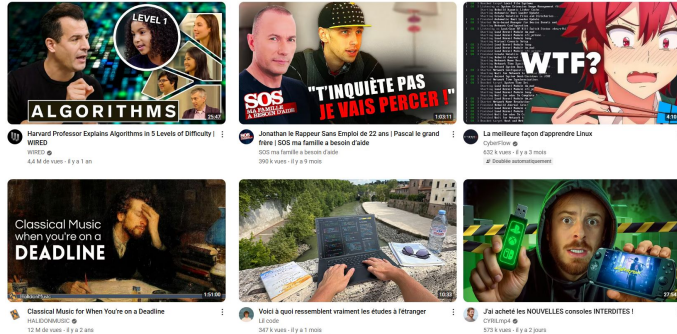


Quel est le problème ?

Systèmes de recommandation

Problème du classement

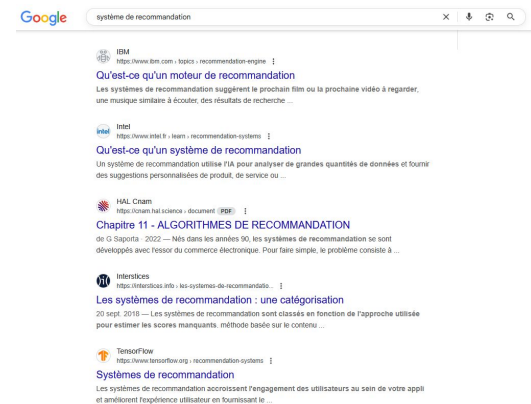
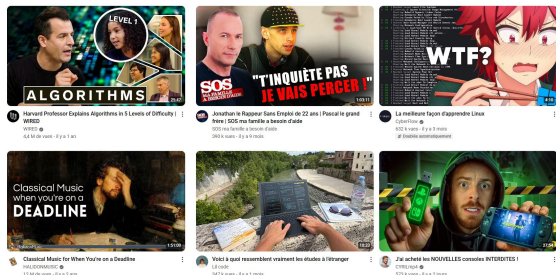
Problème de la prédiction



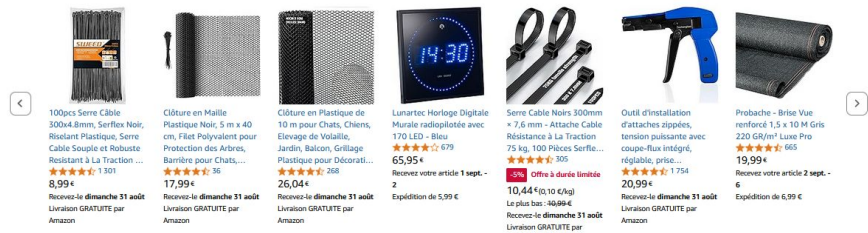
						
	4	?	?	2	3	?
	?	2	?	4	?	5
	4	5	?	?	4	?
	2	?	3	?	2	?
	5	4	2	5	?	4
	?	?	3	?	2	?

Problème du classement

- À partir d'un ensemble d'éléments donné, on cherche à recommander un sous-ensemble d'éléments pertinents.
- Pour y parvenir, on peut utiliser différentes informations (e.g. popularité, notes, comportement des utilisateurs, etc.).
- Classiquement** : contenu à regarder, articles intéressants dans une boutique, prochaine vidéo ou musique, résultat d'une recherche, etc.



Les clients ayant acheté cet article ont également acheté



Problème de la prédiction

- On élargit le problème par rapport à un simple classement : cette fois on essaye de prédire **toutes les combinaisons utilisateurs/éléments**.
- On considère une **matrice**, où **chaque ligne représente un utilisateur et chaque colonne représente un élément**. La valeur de la cellule (i, j) représente une mesure de « **pertinence** » de l'élément j pour l'utilisateur i .
- Cette mesure de « **pertinence** » peut être variée : note, like, visionnage, réactions, etc.
- Le problème de la prédiction est, *a priori*, **plus difficile que le problème du classement**. Il est aussi plus **général** : on peut faire des recommandation pour n'importe quel utilisateur à partir d'une solution complète.



	4	?	?	2	3	?
	?	2	?	4	?	5
	4	5	?	?	4	?
	2	?	3	?	2	?
	5	4	2	5	?	4
	?	?	3	?	2	?

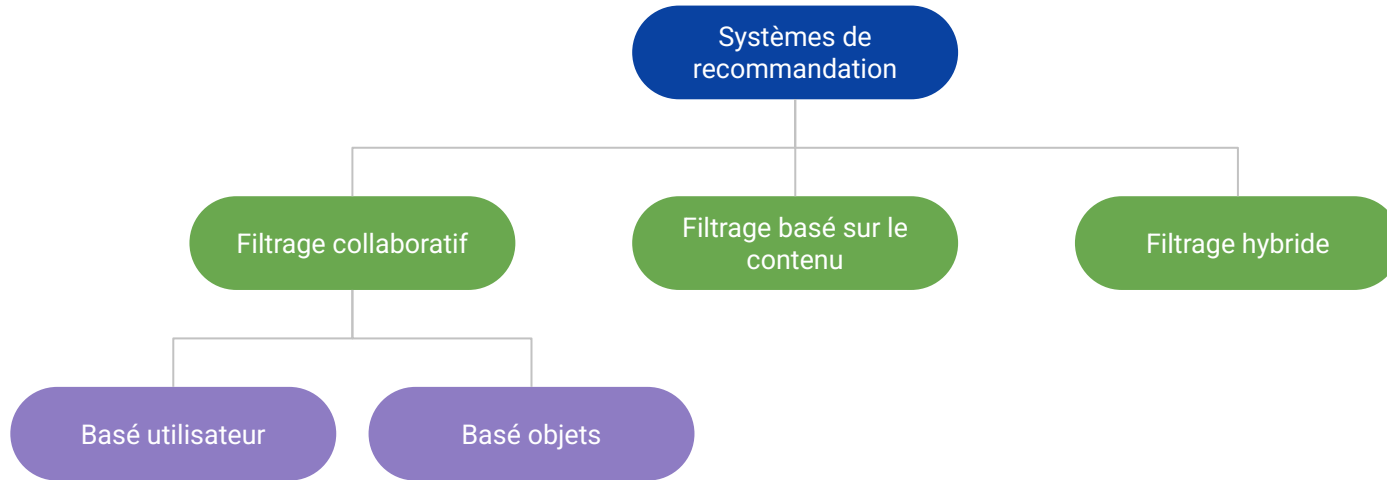
Le problème de la « pertinence »



- Choisir comment mesurer la pertinence (*feedback*) est **primordial** ! Cela revient à décider de l'objectif du système de recommandation.
- On peut opter pour une approche:
 - **explicite** : l'utilisateur donne explicitement son avis (e.g. *like*, note, réaction) ;
 - **implicite** : on utilise une information d'interaction obtenue sans que l'utilisateur n'ait donné explicitement son avis (e.g. achat, *watch time*) ;
 - **hybride** : on associe plusieurs mesures explicites et implicites.
- **Enjeux opérationnels** important pour le système (préférences utilisateurs ? rentabilité ? confidentialité ?)

Problème du classement

Les approches classiques



Filtrage collaboratif

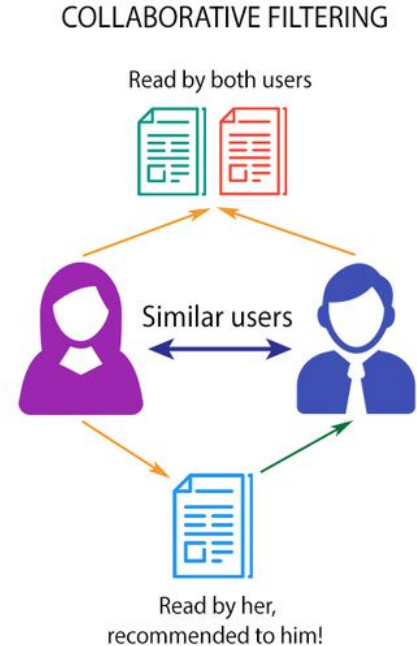
Définition du filtrage collaboratif

Idée

- La technique consiste à exploiter les **similarités** entre les comportements des utilisateurs, calculées sur une mesure de pertinence.
- **L'objectif** est de recommander à l'utilisateur un élément qui a été apprécié (ou considéré pertinent) par **d'autres utilisateurs qui lui ressemblent**.

Remarque

- Cette méthode ne nécessite que des **informations d'utilisation** sur les éléments. **Il n'est pas nécessaire d'avoir accès à d'autres informations** (e.g. métadonnées).



Comment mesurer la similarité



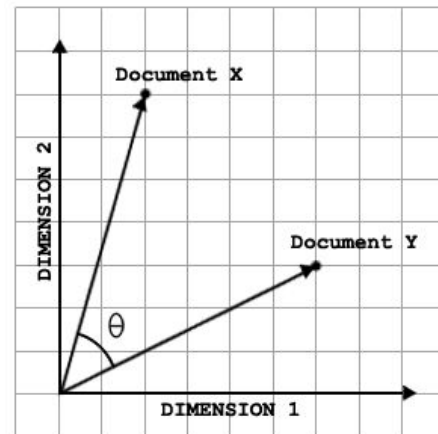
- **Métrique de similarité**
 - *Fonction à valeurs réelles qui permet de quantifier la similarité entre deux objets.*
 - En particulier dans notre cas : entre deux vecteurs.
- **Exemples**
 - Distance euclidienne : longueur du segment entre les points.
 - Coefficient de corrélation de Pearson : mesure de la corrélation linéaire.
 - Distance de Jaccard (IoU) : mesure du nombre d'éléments en commun.
- Généralement dans les système de recommandation, on utilise la **similarité cosinus**.

Similarité cosinus

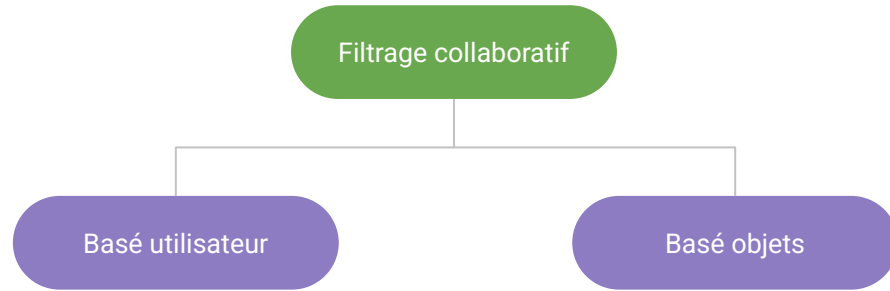
- On considère les éléments comme des vecteurs dans un repère, et on s'intéresse à l'angle qu'il y a entre eux :
 - si cet angle est faible, les vecteurs pointent dans la même direction, les éléments sont proches ;
 - à l'inverse, si cet angle est grand, les éléments sont plus éloignés.
- Plutôt que de considérer l'angle θ directement, on peut considérer $\cos(\theta)$ (grâce à la formule du produit scalaire).

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \cdot \sqrt{\sum_{i=1}^n B_i^2}}$$

- Cette valeur va varier entre -1 et 1 .
 - $\theta = 0^\circ, \cos(\theta) = 1$;
 - $\theta = 180^\circ, \cos(\theta) = -1$;
 - $\theta = 90^\circ, \cos(\theta) = 0$.



Les approches collaboratives



On sélectionne les utilisateurs qui sont similaires à moi, et on me recommande les éléments qu'ils ont aimé.

On recommande les éléments qui ont été aimés ensemble (e.g. les utilisateurs qui ont acheté X, ont aussi acheté Y)

Filtrage collaboratif basé utilisateur

Principe

On cherche à faire une recommandation à un utilisateur, en regardant ce que les utilisateurs similaires ont apprécié.

Exemple

 et  sont très proches.

 et  sont un peu moins proches.

Quel film recommander à  ?

						
Alice 	4			2	3	
Bob 		2		4		5
Kim 	4	5			4	3
Bernard 	2		3		2	0
Mallory 	5	4	2	5		4
Igor 			3		2	

Filtrage collaboratif basé utilisateur

En utilisant la similarité cosinus, on peut calculer quantitativement la « proximité » entre toutes les paires d'utilisateurs (chaque utilisateur étant représenté par un vecteur de ses notes).

Problème 🤔 :

Certaines valeurs sont manquantes (tout le monde n'a pas tout vu, c'est le principe !), mais la formule de la similarité exige une valeur.

Une solution triviale consiste à mettre des 0 lorsque la valeur est manquante.

Mais comment faire mieux ?

						
Alice 	4	0	0	2	3	0
Bob 	0	2	0	4	0	5
Kim 	4	5	0	0	4	3
Bernard 	2	0	3	0	2	0
Mallory 	5	4	2	5	0	4
Igor 	0	0	3	0	2	0

Similarité cosinus

Filtrage collaboratif basé utilisateur

Exemple

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \cdot \sqrt{\sum_{i=1}^n B_i^2}}$$

SimCos(, ) =

$$(4 \times 4 + 3 \times 4) / [\sqrt{(4^2 + 2^2 + 3^2)} \times \sqrt{(4^2 + 5^2 + 4^2 + 3^2)}]$$
$$= 28 / (\sqrt{29} \times \sqrt{66})$$
$$\approx 0,64$$

En faisant ça pour toutes les combinaisons, on obtient une **matrice de similarité** des utilisateurs.

1. *Que remarquez-vous ?*
2. *Comment utiliser cette matrice pour faire une recommandation ?*

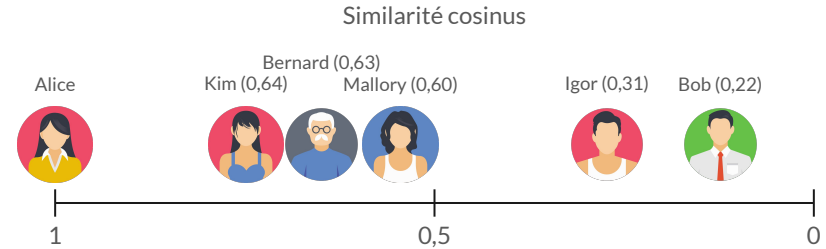
						
Alice	1	0,22	0,64	0,63	0,60	0,31
Bob	0,22	1	0,46	0	0,77	0
Kim	0,64	0,46	1	0,48	0,69	0,27
Bernard	0,63	0	0,48	1	0,42	0,87
Mallory	0,60	0,77	0,69	0,42	1	0,18
Igor	0,31	0	0,27	0,87	0,18	1

Filtrage collaboratif basé utilisateur

Exemple

Pour faire une recommandation à .

On commence par regarder les utilisateurs proches :



On conserve alors les classements des utilisateurs les plus proches, disons **Kim et Bernard**.

On peut alors faire une moyenne pondérée de leurs avis, **selon la proximité avec Alice**.



Filtrage collaboratif basé utilisateur



Algorithme

1. Si on ne l'a pas déjà, on calcule la **matrice de similarité** entre tous les utilisateurs.
2. On sélectionne les **k plus proches voisins** de l'utilisateur à qui on veut faire une recommandation.
3. On récupère les **éléments déjà vus par les voisins**, et qui **n'ont pas été déjà vus par l'utilisateur**.
4. On calcule la **moyenne pondérée de ces éléments**, en utilisant les notes des voisins pondérée par le coefficient de similarité.
5. On peut alors recommander ces éléments en utilisant la note prédite pour les classer.

Questions

- *Comment choisir k ?*
- *Combien de recommandations faut-il afficher à l'utilisateur ?*
- *Comment pourrait-on intégrer du machine learning dans un tel système ?*

Filtrage collaboratif basé objet

Principe

Même principe que le basé utilisateur, mais cette fois on regarde les films qui ont été appréciés de la même façon.

Exemple



et sont souvent aimés ensemble.



et ne sont pas souvent aimés ensemble.



Alice		4			2	3	
Bob			2		4		5
Kim		4	5			4	3
Bernard		2		3		2	0
Mallory		5	4	2	5		4
Igor				3		2	

Quel film recommander à



Filtrage collaboratif basé objet

Encore une fois, on peut utiliser la similarité cosinus, mais cette fois entre les films, c'est-à-dire **entre les colonnes**.

Même problème 🤔 : cette fois encore, il nous faut compléter les valeurs manquantes.

						
Alice 	4	0	0	2	3	0
Bob 	0	2	0	4	0	5
Kim 	4	5	0	0	4	3
Bernard 	2	0	3	0	2	0
Mallory 	5	4	2	5	0	4
Igor 	0	0	3	0	2	0

↑
Similarité cosinus
↑

Filtrage collaboratif basé objet

Exemple

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \cdot \sqrt{\sum_{i=1}^n B_i^2}}$$

SimCos(, ) =

$$(2 \times 5 + 5 \times 3 + 4 \times 4) / [\sqrt{(2^2 + 5^2 + 4^2)} \times \sqrt{(5^2 + 3^2 + 4^2)}]$$

$$= 41 / (\sqrt{45} \times \sqrt{50})$$

$$\approx 0,86$$

En faisant ça pour toutes les combinaisons, on obtient une **matrice de similarité** des objets.

Comment faire une recommandation à partir de cette matrice ?

					
1	0,76	0,44	0,63	0,71	0,58
0,76	1	0,25	0,62	0,52	0,86
0,44	0,25	1	0,32	0,45	0,24
0,63	0,62	0,32	1	0,16	0,84
0,71	0,52	0,45	0,16	1	0,30
0,58	0,86	0,24	0,84	0,30	1

Filtrage collaboratif basé objet

Exemple

Pour faire une recommandation à .

On commence par regarder les films qu'elle a aimés :

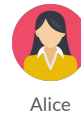
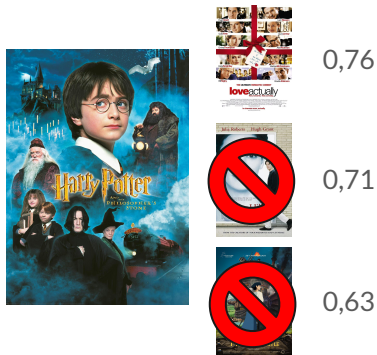


4/5



3/5

Pour chacun de ces films, on sélectionne disons les trois films qui ont été **le plus appréciés de la même façon**. On peut ensuite faire une moyenne pondérée de la note d'Alice, **selon la proximité des films**.



Alice



$$\Rightarrow (0,76 \times 4 + 0,52 \times 3) / (0,76 + 0,52) = 3,6$$

$$\Rightarrow (0,45 \times 3) / (0,45) = 3,0$$



Déjà vu !

Filtrage collaboratif basé objet



Algorithme

1. Si on ne l'a pas déjà, on calcule la **matrice de similarité** entre tous les éléments (« objets »).
2. On sélectionne, pour l'utilisateur à qui on veut faire une recommandation, les **n éléments qu'il a le plus appréciés**.
3. Pour chacun de ces éléments, on sélectionne les **k plus proches voisins**, qui **n'ont pas été déjà vus par l'utilisateur**.
4. On calcule la **moyenne pondérée de ces éléments**, en utilisant la note de l'utilisateur à l'élément qu'il a aimé pondérée par le coefficient de similarité avec l'élément.
5. On peut alors recommander ces éléments en utilisant la note prédite pour les classer.

Questions

- *Quelle est la différence entre n et k ?*
- *Quelle approche vous paraît être la meilleure entre le filtrage collaboratif basé utilisateur, ou basé objet ?*

Filtrage collaboratif, basé utilisateur VS basé objet



- Les résultats peuvent être différents (et généralement, ils le sont !).
- **Sur le plan technique** : le coût de calcul de la matrice de similarité va croître soit en fonction du nombre d'utilisateurs (basé utilisateur), soit en fonction du nombre d'éléments (basé objet). Le coût en calculs et en mémoire n'est donc pas forcément le même.
- **Sur le plan pratique** : similaire dans son principe, mais objectif différent
 - **Pour le basé utilisateur** : « je te recommande les éléments que les utilisateurs qui te ressemblent ont aimé ».
 - **Pour le basé objet** : « je te recommande les éléments qui ont été aimés de la même façon que ceux que tu as déjà aimé »
- *Dans la vraie vie, quels sont les cas où l'une ou l'autre de ces approches vous paraissent pertinentes ?*

Filtrage collaboratif (FC) : avantages et inconvénients



Avantages

- **Cadre générique** : l'approche fonctionne avec une très grande variété de métriques, on peut même les combiner assez facilement (exemple : *watch time* + interactions + *likes*).
- Fonctionne sur **n'importe quels éléments**, dès lors qu'on a des données d'utilisation.
- **Sérendipité naturelle** : possibilité de faire des recommandations auxquelles l'utilisateur ne s'attend pas.

Inconvénients

- Il faut des **données d'utilisation** pour faire du FC (problème de confidentialité, ou de faisabilité).
- **Cold-start problem** : il faut des données d'utilisation pour faire du FC avant de pouvoir faire une recommandation à un nouvel utilisateur.
- **Biais de popularité** : les éléments peu populaires ne seront pas recommandés, même s'ils sont très pertinents dans l'absolu.
- **Explicabilité limitée** : il est difficile d'apporter une explication sur les recommandations à l'utilisateur (surtout sur du FC basé utilisateur, moins sur du FC basé objet).

Filtrage basé sur le contenu

Définition du filtrage basé sur le contenu

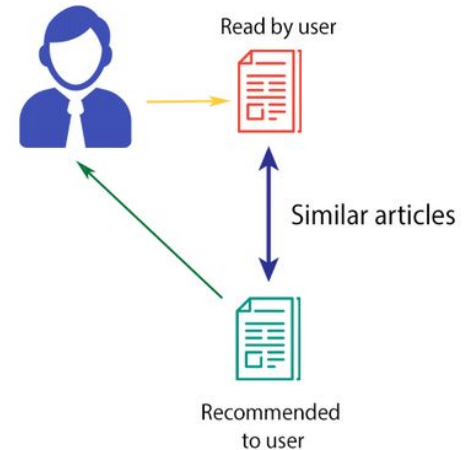
Idée

- La technique consiste à exploiter les **similarités** entre les éléments à recommander **eux-mêmes**, sans considération pour le comportement des autres utilisateurs.
- **L'objectif** est de recommander à l'utilisateur un élément qui ressemble à des éléments qu'il a déjà aimé.

Remarque

- À l'inverse du filtrage collaboratif, cette méthode ne nécessite que des **informations sur les éléments eux-mêmes** (e.g. titre, contenu, métadonnées). **Il n'est pas nécessaire d'avoir accès à d'autres informations** (e.g. données d'utilisation d'autres utilisateurs).

CONTENT-BASED FILTERING



Source: L. Tondji, [Web Recommender System for Job Seeking and Recruiting](#)

Le problème de la comparaison



Difficulté : comment faire pour comparer les éléments entre eux ?

Par exemple, comment dire que tel livre est proche de tel autre livre, ou que tel film est proche de tel autre film ?

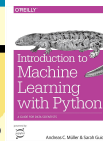
Solutions

- Choisir les éléments de comparaison en fonction du contenu des éléments. Il faut que ce soit représentatif !
 - Titre
 - Métadonnées : auteur, acteurs, durée, type, date, etc.
 - Contenu
- Contrainte de la représentation : il faut pouvoir représenter les éléments de façon à pouvoir les comparer **quantitativement**.
 - Nombre ➡ on sait déjà comparer des vecteurs de nombres 🥳 !
 - Texte ➡ *comment faire ?*
 - Image ? Vidéo ? ➡ *est-ce possible ?*

Exemple

Considérons un ensemble de **livres**, ainsi qu'une **utilisatrice** qui a déjà des préférences.

Alice 

					
1		3			5

Comment générer des recommandations pour cet exemple ?

Exemple avec des données textuelles

Idee : nous pourrions, par exemple, représenter les livres par leur titre. Cela permettrait d'exploiter la représentation textuelle.



Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow



Python Crash Course



Introduction to Machine Learning with Python



Introduction to Algorithms



Machine Learning with PyTorch and Scikit-Learn



Data Structures & Algorithms in Python

On peut ensuite utiliser des techniques d'analyse de texte pour comparer les livres.

Exemple : transformer les titres en vecteur de mots.

Exemple avec des données textuelles



Algorithms				1	1
Course		1			
Crash		1			
Data					1
Hands-on	1				
Introduction			1	1	
Keras	1				
Learning	1		1		1
Machine	1		1		1
PyTorch		1			1
Python		1	1		1
Scikit-Learn	1				1
Structures					1
TensorFlow	1				

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow

Python Crash Course

Introduction to Machine Learning with Python

Introduction to Algorithms

Machine Learning with PyTorch and Scikit-Learn

Data Structures & Algorithms in Python

Comment utiliser cette représentation vectorielle pour faire une recommandation ?

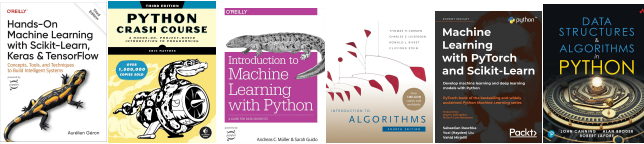
Exemple avec des données textuelles



Algorithms				1		1
Course	1					
Crash	1					
Data						1
Hands-on	1					
Introduction			1	1		
Keras	1					
Learning	1		1		1	
Machine	1		1		1	
PyTorch		1			1	
Python		1	1			1
Scikit-Learn	1				1	
Structures						1
TensorFlow	1					

↑
Similarité cosinus

On peut calculer la matrice de similarité à partir de la représentation vectorielle :



	1	0	0,41	0	0,61	0
	0	1	0,25	0	0,25	0,25
	0,41	0,25	1	0,35	0,5	0,25
	0	0	0,35	1	0	0,35
	0,61	0,25	0,50	0	1	0
	0	0,25	0,25	0,35	0	1

Exemple avec des données textuelles

Pour faire une recommandation à



		3			5

Alice



Pour chaque livre non-noté par Alice, on peut calculer une **moyenne pondérée des notes d'Alice sur les livres notés**, selon leur coefficient de proximité avec le livre considéré.



Des comparaisons plus sophistiquées

Nous avons utilisé une représentation et une comparaison très simple pour cet exemple, mais il est possible d'opter pour des solutions plus puissantes ou complètes.

- Techniques de fouille de données
 - e.g. TF-IDF

Exemple

$TF(t, d)$ = la fréquence relative du terme t dans le document d

$IDF(t) = \log(N/n_t)$ avec :

- N le nombre de documents dans le corpus
- n_t est le nombre de documents du corpus où le terme t apparaît

$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$

Version lissée

$$\log\left(\frac{N}{1 + n_t}\right) + 1$$

- Machine learning
 - e.g. modèle entraîné pour comparer des éléments

Filtrage basé contenu : avantages et inconvénients



Avantages

- Il n'est **pas nécessaire de disposer de données d'utilisation** (intérêt de confidentialité, ou de réglementation).
- **Cold-start problem** : il n'est pas nécessaire d'avoir beaucoup d'information sur un nouvel utilisateur avant de pouvoir lui faire des recommandations.
- Pas de **biais de popularité**.
- Généralement **explicable** (mais ce n'est pas toujours le cas).

Inconvénients

- **Souvent très spécifique** : la représentation et la méthode de comparaison choisies doivent être parfaitement adaptées aux données disponibles, elle sont plus difficilement réutilisables.
- **Sérendipité faible** : par définition, seuls les éléments similaires seront recommandés, les découvertes étonnantes heureuses sont donc beaucoup moins probables.
- Il n'est **pas aisé de combiner les données** sur le contenu (e.g. titre + acteurs + vidéo), il faut généralement se limiter à une ou plusieurs modalités simples et compatibles.

Filtrage hybride et plus encore !

Filtrage hybride



Idée

- On combine le principe du filtrage collaboratif avec le filtrage basé contenu pour essayer d'avoir un compromis entre les avantages et les inconvénients de chaque approche.

Méthodes (typiques)

- **Pondération** : les prédictions de plusieurs systèmes de recommandation sont moyennées, avec des coefficients choisis.
- **Commutation** : selon les circonstances, l'un ou l'autre des systèmes est utilisé pour faire la recommandation (par exemple : nouvel utilisateur vs utilisateur ancien).
- **Mix** : les différentes recommandations des différents systèmes sont présentées en même temps à l'utilisateur (typiquement : Netflix).
- **Cascade** : la sortie d'un système de recommandation est donnée en entrée à un autre système.
- *Etc. The limit is your imagination!* 🌈

Knowledge-based filtering



Problème

Dans certains cas, le système de recommandation que l'on cherche à développer s'inscrit dans un domaine très spécifique :

- **contexte précis et complexe** : sujets techniques où l'utilisateur a déjà des préférences ou des critères assez précis, cela nécessite d'avoir de l'information supplémentaire pour être pertinent ;
- **peu de données d'utilisation** : typiquement lorsque cela concerne des éléments qui ne sont pas « consommés » ou « recherchés » souvent.

À quoi vous pensez ?

Exemples classiques : immobilier, informatique, photographie, voiture, tourisme, etc.

Knowledge-based filtering

Solution du *knowledge-based*

Le système de recommandation acquiert spécifiquement l'information dont il a besoin avant de faire sa recommandation. Plusieurs approches sont possibles :

- Critères explicites de l'utilisateur
- Approche conversationnelle (généralement via une *feedback loop*)

FILTRE LES PRODUITS :

RÉINITIALISER LES FILTRES

CHERCHER UNE RÉFÉRENCE

Désignation, modèle ...

MARQUE

Sélectionner

PRIX

799 1500

TAILLE DE L'ÉCRAN

13 pouces, 14 pouces, 11 pouc...

PROCESSEUR

AMD Ryzen 7

CHIPSET GRAPHIQUE

AMD Radeon Graphics

SYSTÈME D'EXPLOITATION

Sélectionner

+ DE FILTRES

Voir uniquement les produits

10 PRODUITS CORRESPONDENT

Prix croissant

AMD Radeon Graphics × AMD Ryzen 7 × 11 pouces × 12 pouces × 13 pouces × 14 pouces × 10 pouces ×

☐

ASUS Vivobook 14 S1405YA-LY008W
AMD Ryzen 7 7730U 16 Go SSD 512 Go 14" LED Full HD+ Wi-Fi 6E/Bluetooth Webcam Windows 11 Famille

DISPO
Web
EN STOCK
799€⁹⁵
Dispo dans 1 boutique

☐

ASUS Vivobook 14 S1407KA-LY054W Copilot+ PC
AMD Ryzen AI 7 350 16 Go SSD 512 Go 14" LED Full HD+ Wi-Fi 6E/Bluetooth Webcam Windows 11 Famille

DISPO
Web
EN STOCK
919€⁹⁵

☐

PROMOTION
ASUS Vivobook S 14 OLED S5406UA-PP003W
AMD Ryzen 7 8845HS 16 Go SSD 1 To 14" OLED 3K 120 Hz Wi-Fi 6E/Bluetooth Webcam Windows 11 Famille
★★★★★ 1 avis

DISPO
Web
EN STOCK
~~1 199€⁹⁵~~
999€⁹⁵

☐

ASUS Vivobook S 14 OLED M5406UA-QD093X
AMD Ryzen 7 8845HS 16 Go SSD 1 To 14" OLED Full HD+ Wi-Fi 6E/Bluetooth Webcam Windows 11 Professionnel

DISPO
Web
EN STOCK
1 079€⁹⁵

Problème de la prédiction

Problème de la prédiction

- Cette fois, on ne se limite pas à donner des recommandation pour un utilisateur, on veut complètement remplir la matrice !
- Le problème de la prédiction est, *a priori*, **plus difficile que le problème du classement**. Il est aussi plus **général** : une fois résolu, on peut faire des recommandation pour n'importe quel utilisateur à partir d'une solution complète.
- Les approches vues précédemment (filtrages collaboratif, basé sur le contenu, hybride) restent valables, mais il faut les étendre.



	4	?	?	2	3	?
	?	2	?	4	?	5
	4	5	?	?	4	?
	2	?	3	?	2	?
	5	4	2	5	?	4
	?	?	3	?	2	?

Approche basée sur un modèle

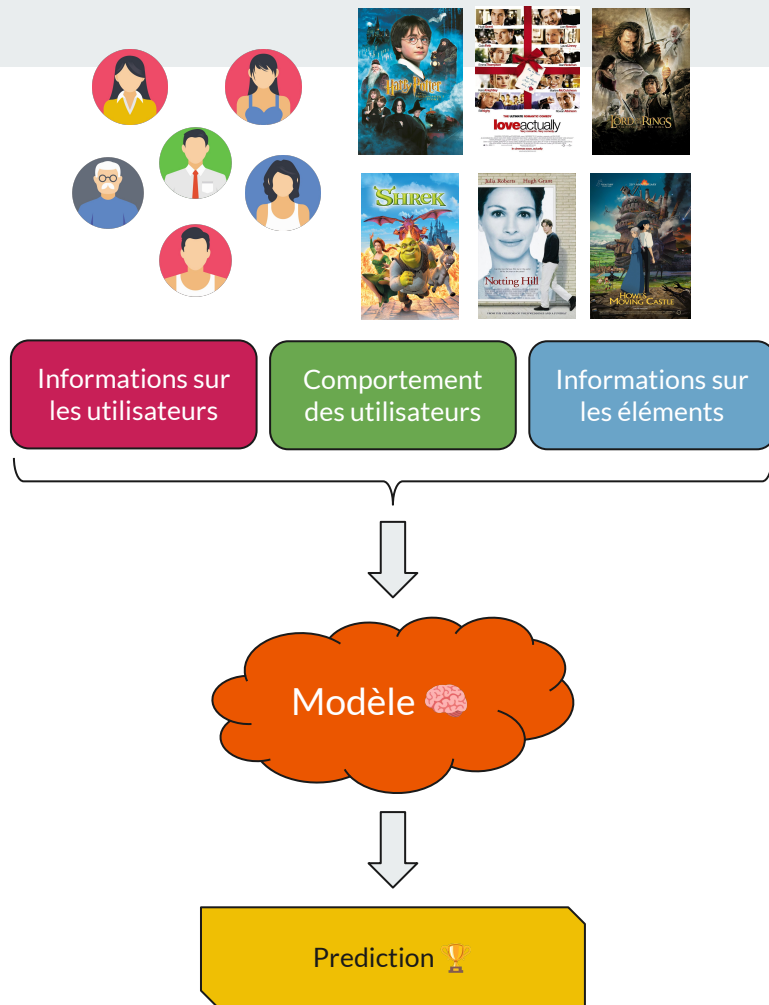
La solution classique au problème de la prédiction est une **approche basée sur un modèle** (*model-based method*).

Principe

On conçoit un modèle qui va apprendre à partir des données à notre disposition, de façon à pouvoir ensuite généraliser et prédire toutes les combinaisons.

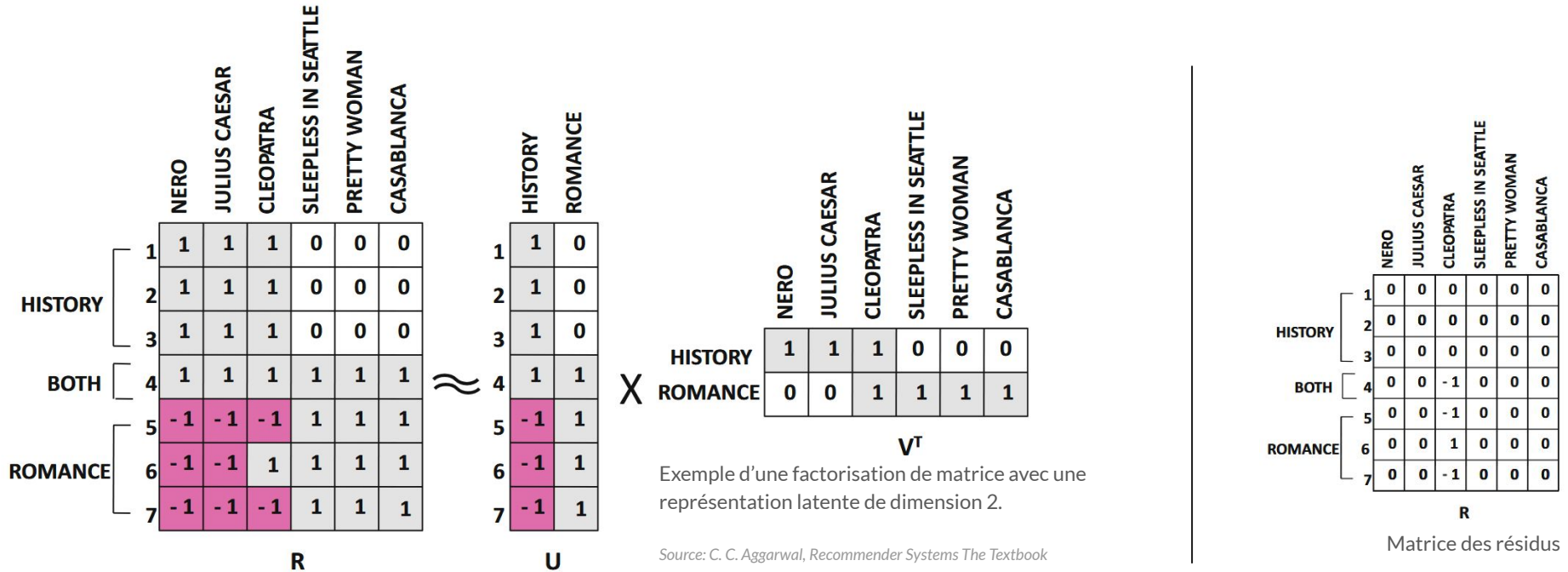
Modèles usuels

- Factorisation matricielle
- *Machine learning*



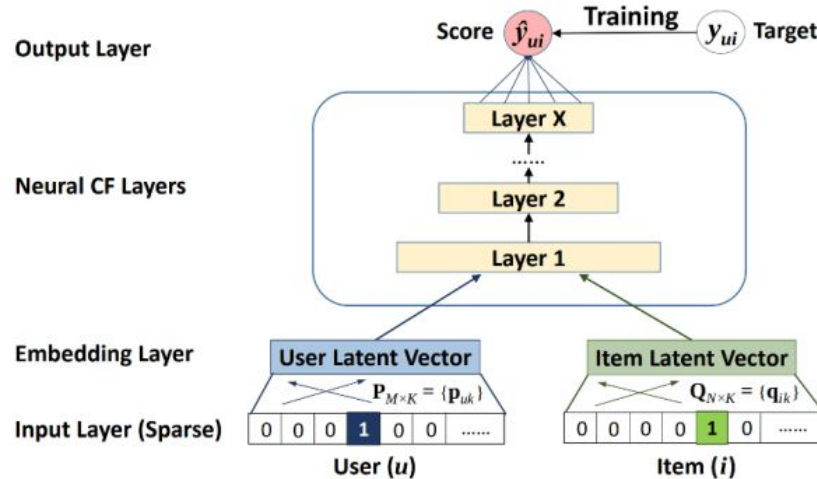
Factorisation matricielle

On fait l'hypothèse de l'existence d'une **représentation latente** explicative d'une certaine dimension. On cherche ensuite à factoriser la matrice de pertinence dans cette dimension.



Machine learning

On entraîne un modèle sur les données existantes pour prédire les données manquantes (apprentissage supervisée).



Exemple d'un réseau de neurones de type *Neural Collaborative Filtering* pour le problème de la prédiction

Conseils pratiques

Comment concevoir mon système de recommandation ?



- Avant de vous concentrer sur la technique, **ne négligez pas la phase de réflexion** autour de votre système.
 - Quel est l'objectif de mon système de recommandation ?
 - Qu'est-ce qui est une bonne recommandation dans mon cas ?
 - Lien avec ma stratégie commerciale si j'en ai une
 - Quelles données j'ai ou je peux avoir à disposition ?
 - Lesquelles je veux utiliser pour mon système ?
- Essayez d'avancer itérativement. Testez vos différentes approches !
 - Comment mesurer l'efficacité de mon système ?
 - A/B testing
- Soyez critique mais constructif : réfléchissez aux faiblesses usuelles et à l'impact qu'elles peuvent avoir sur votre système.

Bonnes recommandations à tous !

Des questions ? 😊

